

Rapport TP3 : Redis et caches distribués

Partie 2 — Documentation du gain observé (Cache-aside)

Dans le script Python fourni, nous avons simulé une base de données avec une latence artificielle de 10 ms. * **Temps de lecture à froid (Cache Miss)** : La première lecture prend un peu plus de 10 ms (le temps de consulter la BDD, puis d'écrire la donnée en cache dans Redis). * **Temps de lecture à chaud (Cache Hit)** : Lorsqu'on boucle 1000 fois sur la fonction de lecture pour le même produit, la requête ne va plus solliciter la base de données. Redis servant les données depuis la RAM en moins d'une milliseconde, les 1000 itérations s'exécutent généralement en une fraction de seconde (souvent entre 5 et 15 ms au *total* pour les 1000 requêtes en local). * **Conclusion** : Le gain de performance est gigantesque. Le cache permet de décharger la base de données principale (MongoDB ou SQL) des requêtes de lecture répétitives, et divise la latence par 10 ou plus pour l'utilisateur final.

Partie 4 — Questions de cours

1. Pourquoi Redis n'est-il pas adapté comme unique base de persistance ?

Redis est une base de données "In-Memory", ce qui signifie que l'intégralité du dataset doit tenir dans la RAM du serveur. La RAM étant extrêmement plus coûteuse au gigaoctet que le stockage disque (SSD/HDD), héberger un grand volume de données relationnelles ou d'archives (tébioctets) sur Redis coûterait une fortune. De plus, bien qu'il possède des mécanismes de persistance (AOF/RDB), son architecture n'est pas taillée pour les requêtes complexes (comme les jointures SQL ou les agrégations profondes sur de larges datasets), mais plutôt pour des accès simples et ultra-rapides par clé.

2. Que se passe-t-il si Redis redémarre sans persistance (RDB/AOF) ?

Si la persistance RDB (snapshots) et AOF (Append Only File) sont toutes deux désactivées, Redis agit comme un cache volatil pur. Si le processus Redis crashe ou que le serveur redémarre (ou si le conteneur Docker est recréé sans volume persistant), **toutes les données en mémoire sont définitivement perdues**. Lors du redémarrage, la base Redis sera entièrement vide.

3. Différence entre TTL et suppression manuelle du cache ?

- **TTL (Time To Live) :** C'est un mécanisme passif. Lors de l'écriture en cache, on définit une durée de vie (ex: 300 secondes). Redis supprimera automatiquement la clé une fois le temps écoulé. Cela garantit que le cache ne devient pas infini (éviction) et que la donnée finira par être rafraîchie avec la base de données.
- **Suppression manuelle (Cache Invalidation) :** C'est un mécanisme actif (comme notre commande `r.delete` dans `update_product`). Dès qu'une donnée est modifiée en base (mise à jour d'un prix), l'application supprime immédiatement la clé correspondante dans Redis. Cela permet d'éviter que l'utilisateur continue de voir l'ancien prix pendant les 300 secondes du TTL. Les deux mécanismes sont souvent utilisés ensemble : la suppression manuelle pour la cohérence en temps réel, et le TTL en filet de sécurité au cas où l'événement d'invalidation aurait échoué.